

Lab1

Task1.1:

Relation A:

1. 6 different superkeys:
EmpID, SSN, Email, Phone, {EmpID, Name}, {SSN, Department}
2. All candidate keys:
EmpID, SSN, Email, Phone
3. EmpID, It is a synthetic/surrogate attribute specifically designed as a unique identifier that never changes, avoiding issues with personal privacy (SSN) or potential updates to contact info (Email, Phone).
4. Based strictly on the provided data, no (all numbers in the sample are unique: 555-0101, 555-0102, 555-0103). However, in real-world scenarios and database design principles, two employees could potentially share a corporate phone line, so Phone should not be relied upon as a permanent unique identifier.

Relation B:

1. {StudentID, CourseCode, Section, Semester, Year}
2.
 1. StudentID: Identifies which student is registering.
 2. CourseCode and Section: Identify the specific class/offering being taken.
 3. Semester and Year: Ensure a student can retake the exact same course section in a different semester or year (fulfilling the business rule that a student can take the same course in different semesters).
3. None, because no other minimal subset of attributes uniquely identifies a registration record without redundancy.

Task1.2:

Student.AdvisorID to Professor.ProfID

Professor.Department to Department.DeptCode

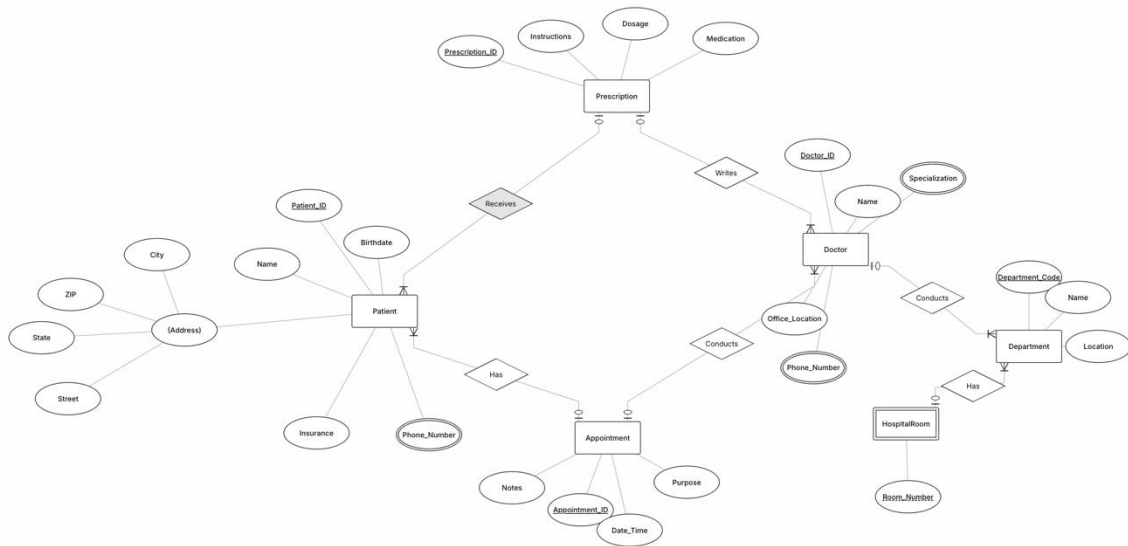
Course.DepartmentCode to Department.DeptCode

Department.ChairID to Professor.ProfID

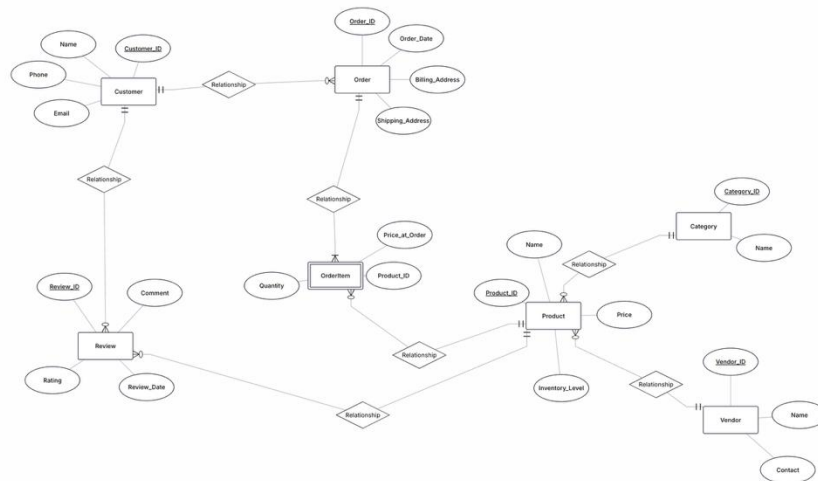
Enrollment.StudentID to Student.StudentID

Enrollment.CourseID to Course.CourseID

Task 2.1:



Task 2.1:



Task 4.1:

1. Functional Dependencies:

Student ID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, Supervisor ID, StartDate, EndDate

Supervisor ID $\rightarrow$ SupervisorName, SupervisorDept

Student ID, ProjectID $\rightarrow$ Role, HoursWorked

2. Problems:

Redundancy: Student details, project information, and supervisor data are duplicated across multiple rows.

Anomalies: Update, insert, and delete anomalies occur due to multiple unrelated attributes depending on different keys within the same table.

3. 1NF:

Ensure all values are atomic and every table has a primary key.

4. 2NF:

Primary Key: (Student ID, ProjectID)

Partial Dependencies: Attributes that depend only on part of the primary key (Student ID or ProjectID)

2NF Decomposition: Split into separate tables for Students, Projects, and StudentProjectAssignments

5. 3NF:

Transitive Dependencies: ProjectID $\rightarrow$ Supervisor ID $\rightarrow$ SupervisorName, SupervisorDept

Final 3NF Decomposition: Student, Supervisor, Project, and StudentProjectAssignment

Task 4.2:

1. Composite Primary Key: (Student ID, CourseID, TimeSlot)

2. Functional Dependencies:

Student ID $\rightarrow$ StudentMajor

CourseID $\rightarrow$ CourseName

InstructorID $\rightarrow$ InstructorName

TimeSlot, Room $\rightarrow$ Building

CourseID, TimeSlot $\rightarrow$ InstructorID, Room

3. BCNF:

The table is not in BCNF because non-superkeys determine other attributes.

4. BCNF Decomposition:

Decomposed into Student, Course, Instructor, RoomBuilding, CourseScheduleDetails, and StudentEnrollment.

5. Information Loss:

Using proper lossless-join decomposition with foreign keys ensures no loss of information.

Task 5.1:

